Spesifikasi Tugas Besar

IF2110 Algoritma dan Pemrograman 2

Tim Asisten IF2110 Algoritma dan Pemrograman 2 2025/2026

Versi	: **v1.0**
Tgl. Revisi Terakhir	: **08 November 2025 Pukul 21.10 WIB**
Deadline	: **06 Desember 2025 Pukul 21.10 WIB**
Revisi	**:**

- **[v1.1] 15 November 2025**
  - **Penambahan ID komentar pada contoh perintah "VIEW_POST"**
  - **Perubahan folder contoh konfigurasi atau sample data**
  - **Perbaikan contoh sample data untuk voting (voting.csv)**
  - **Perbaikan Bonus "Advanced Search"**
- **[v1.2] 17 November 2025**
  - **Perbaikan penjelasan konfigurasi social**
- **[v1.3] 02 Desember 2025**
  - **Perbaikan penjelasan fitur bonus data security**
- **[v1.4] 04 Desember 2025**
  - **Perpanjangan tenggat waktu pengumpulan tugas besar**

Daftar Isi

Daftar Isi **2**
Tujuan Tugas Besar **3**
Latar Belakang **4**
Spesifikasi Program **7**
Daftar ADT **8**
Kebutuhan Program **9**
1. Inisialisasi 9
2. Perintah 9
3. Pengguna 10
4. Profil 12
5. Post 13
6. Subgroddit 16
7. Comment and Reply 18
8. Voting 19
Command untuk Post 20
Command untuk Comment 22
Ketentuan Penting 26
9. Social 26
10. Save & Load 27
Konfigurasi Program **31**
Bonus **35**
1. Trending Analysis 35
2. Advanced Search 36
3. Following Recommendation 37
4. Content Moderation 38
5. Data Security 39
6. Kreativitas 40
Batasan **42**
Saran **42**

Tujuan Tugas Besar

Berikut adalah tujuan yang diharapkan setelah mahasiswa berhasil menyelesaikan tugas besar:

1. Mahasiswa mengenali dan memahami konsep dan dasar ADT.
2. Mahasiswa mampu mendesain dan mengimplementasi ADT.
3. Mahasiswa mampu melakukan pemecahan masalah menggunakan ADT yang sesuai.
4. Mahasiswa mampu menggunakan ADT yang telah disediakan oleh pustaka.

Latar Belakang

Di sebuah laboratorium bawah tanah yang sangat rahasia (dan agak berantakan karena terlalu banyak pisang), Gro dan ribuan Nimon sedang sibuk menciptakan rencana kejahatan terbaru. Dari mesin penyusut raksasa, roket ke bulan, hingga sistem penarik bumi. Semuanya tampak hebat sampai satu masalah muncul: tidak ada yang bisa berkomunikasi dengan benar.

Setiap kali Gro memberi instruksi, para Nimon menafsirkannya dengan semangat luar biasa, namun hasilnya adalah bencana. Ketika Gro berkata *"buat senjata penghancur dunia"*, seseorang justru membuat *"senjata penghancur donat."* Saat ia menulis *"aktifkan mode rahasia,"* ada Nimon yang malah menulis ulang file konfigurasi dan membuat laboratorium berbicara dalam bahasa Nimon selama tiga hari.

Gro akhirnya menatap layar penuh pesan typo, emoji pisang, dan stiker Nimon menari. Ia menghela napas panjang. “Aku tidak butuh roket baru,” katanya. “Aku butuh... sistem komunikasi yang tidak meledak.”

Mendengar itu, Dr. Neroifa, ilmuwan paling eksentrik di Gro Industries (dan satu-satunya yang masih punya alis setelah ledakan minggu lalu), mengangkat tangannya. “Aku punya ide, bos,” katanya dengan penuh semangat. “Bagaimana kalau kita buat forum internal? Tempat semua Nimon bisa berbagi ide, berdebat, dan mungkin sedikit bertengkar soal resep smoothie pisang?” Gro menatapnya ragu, lalu berkata, “Selama tidak ada fitur kirim foto kucing bersenjata, lanjutkan.”

Beberapa hari, lima ledakan kecil, dan dua printer terbakar kemudian, lahirlah **Groditt**, singkatan dari ***Gro’s Digital Discussion Terminal***. Sebuah platform komunikasi berbasis CLI (karena UI dianggap terlalu mahal) yang memungkinkan setiap Nimon memposting ide, berkomentar, dan memberi *upvote* atau *downvote* terhadap rencana kejahatan teman-temannya.

Awalnya Groditt hanya digunakan untuk hal-hal penting, seperti laporan eksperimen, status proyek, dan jumlah pisang yang tersisa di kantin. Namun, seperti semua ciptaan di Gro Industries, semuanya keluar kendali dengan sangat cepat**.**

Dalam waktu seminggu, muncul komunitas baru yang disebut SubGroddit, forum-forum tematik di mana para Nimon berdiskusi tentang berbagai topik, seperti:

- *r/StealTheMoonAgain* — strategi mencuri bulan 2.0.
- *r/BananaScience* — riset ilmiah tentang kenapa semua pisang di laboratorium bersinar di kegelapan.
- *r/NimonOfTheMonth* — kompetisi paling panas di dunia bawah tanah.

Thread populer seperti *“Apakah pisang bisa menggantikan bahan bakar roket?”* mendapat ribuan komentar. Sementara itu, Gro, yang awalnya ingin sekadar memantau, mulai kecanduan memberi upvote dari akun rahasianya, **u/DefinitelyNotGro** (yang tidak terlalu rahasia karena ia selalu menulis “IDE INI CUKUP JAHAT, AKU SUKA”).

Seiring waktu, Groditt menjadi lebih dari sekadar forum internal. Villain lain mulai bergabung diam-diam. Ada yang menggunakan alias, ada pula yang langsung

terang-terangan minta verifikasi akun “Penjahat Terverifikasi.” Bahkan agen AVL (Anti-Villain League) ikut menyamar untuk memantau percakapan, meskipun sebagian besar waktu mereka malah ikut berdiskusi tentang cara terbaik menghindari jebakan laser.

Kini Groditt telah menjadi jantung komunikasi dunia penjahat. Sebuah tempat di mana ide-ide gila lahir, rencana jahat dikembangkan, dan argumen tak penting seperti “mana yang lebih jahat: pisang dingin atau pisang hangat?” tidak pernah berakhir.

Dengan sistem reply tree yang tumbuh lebih cepat dari rambut Gru muda, dan follow graph yang memungkinkan para penjahat mengikuti satu sama lain, Groddit menjadi simbol baru kolaborasi — versi jahat dari jejaring sosial modern. Gro sendiri bangga dengan ciptaannya. “Dulu aku mencuri bulan,” katanya dalam konferensi internal, “tapi sekarang aku mencuri waktu kerja Nimon lewat komentar panjang mereka.”

Dan demikianlah, Groditt, platform sosial paling ramai di Groverse, lahir bukan dari rencana jahat, tapi dari kebutuhan sederhana: mengurangi miskomunikasi (dan jumlah ledakan) di laboratorium.

Spesifikasi Program

Buatlah sebuah program simulasi berbasis **CLI** (*command-line interface*). Program ini dibuat dalam bahasa C dengan menggunakan ADT yang sudah kalian pelajari di mata kuliah ini. Kalian boleh menggunakan ADT yang sudah kalian buat untuk praktikum pada tugas besar ini. Daftar ADT dapat dilihat pada bagian Daftar ADT. Kompilasi program **wajib** menggunakan **Makefile** dan **wajib** dapat dikompilasi dan dijalankan pada sistem operasi berbasis **UNIX**. Jika sistem operasi Anda adalah Windows, gunakan WSL untuk memiliki *environment* seperti UNIX. Contoh Makefile telah diberikan di Template pada GitHub Classroom. Perhatikan bahwa Makefile yang diberikan merupakan contoh, Anda dapat membuat Makefile yang lebih sesuai dengan kebutuhan Anda.

Untuk mempermudah pengerjaan tugas besar, kami telah memberikan contoh pada masing-masing spesifikasi fungsional. Namun, keluaran dan masukan program dapat disesuaikan dengan kreativitas masing-masing. Adapun beberapa informasi tambahan terkait contoh yang kami berikan adalah sebagai berikut:

- Teks Normal berarti output
- **Teks Bold** berarti input
- Teks Biru berarti variabel tertentu

Daftar ADT

Anda diwajibkan menggunakan ADT di bawah ini pada fitur-fitur yang Anda rasa sesuai. Anda wajib memberikan alasan atau ***reasoning*** mengapa Anda memilih ADT tersebut untuk menyelesaikan masalah (fitur). Pastikan bahwa ADT yang anda gunakan dapat menyelesaikan persoalan secara **efektif** dan **efisien**. Setiap masalah/fitur dapat menggunakan kombinasi lebih dari satu ADT. Seluruh ADT yang dibuat wajib dilakukan pengujian. Silahkan membuat *driver* untuk melakukan *unit-testing* pada masing-masing ADT.

1. ADT Sederhana
2. ADT List Berkait
3. Mesin Karakter dan Mesin Kata
4. ADT Tree
5. ADT Graph

Kebutuhan Program

1. Inisialisasi

Program harus melakukan inisialisasi dengan melakukan *load* dari file konfigurasi ketika pertama kali menjalani program. Berikut adalah contoh tampilan program ketika inisialisasi.

```
Selamat datang di Groddit!

Masukkan folder konfigurasi untuk dimuat: config-1;
File konfigurasi berhasil dimuat!

>>
```

2. Perintah

a. Agar pengguna dapat berinteraksi dengan program CLI, diperlukan pembacaan perintah.
b. Pembacaan perintah **wajib** menggunakan **mesin kata**.
c. Segala masukan lainnya juga **wajib** menggunakan **mesin kata**.
d. Mark dari semua perintah dan masukan adalah **; (titik koma).**
e. Setiap masukan harus divalidasi. Sebagai contoh, apabila aplikasi meminta integer, pastikan masukan yang diberikan berupa angka. Jika terdapat huruf atau karakter lain, aplikasi akan memberikan keluaran bahwa masukan tidak valid.

3. Pengguna

a. Untuk dapat menggunakan program dan perintah-perintah yang tersedia, pengguna Groddit harus melakukan **`REGISTER`** dan **`LOGIN`**. Pengguna yang sudah di-autentikasi dapat melakukan **`LOGOUT`**.

b. Sebelum terautentikasi, pengguna tidak dapat melakukan perintah lain selain, **`REGISTER, LOGIN, QUIT`**

c. Setiap pengguna harus memiliki identifier (ID), *username*, dan kata sandi.

- Format untuk ID harus dalam bentuk "USER001", di mana "USER" adalah prefiks tetap dan "001" mewakili urutan angka. Setiap pengguna baru akan di-*assign* ID secara *incremental* oleh sistem.
- *Username* bersifat unik dan panjang maksimumnya adalah 255 karakter.
- Panjang kata sandi **minimum** adalah **8 karakter** dan **maksimum** adalah **20 karakter**.
- Saat melakukan **`REGISTER`**, pengguna memasukkan username dan kata sandi yang diinginkan. Pastikan bahwa setiap pengguna mempunyai *username* yang berbeda.

Perintah	Contoh Program	Keterangan

`REGISTER`	`>> REGISTER;`
	`Masukkan username:`
	`kebinganteng;`
	`Masukkan kata sandi:`
	`P@ssw0Rd;`
	
	`Akun dengan username kebinganteng berhasil didaftarkan! Silahkan gunakan perintah LOGIN untuk mengakses fitur-fitur Groddit`
	
	`>> REGISTER;`
	`Masukkan username:`
	`kebinganteng;`
	`Maaf, username kebinganteng sudah terdaftar :(. Harap pilih username yang lain.`
		Setiap pengguna harus memiliki username unik

LOGIN	>> **LOGIN;** Masukkan *username*: **kebinganteng;** Masukkan kata sandi: **P@ssw0Rd;** Login berhasil, selamat datang di Groddit! <Tampilan Utama> >> **REGISTER;** Anda tidak dapat melakukan registrasi karena telah login sebagai **kebinganteng** >> **LOGIN;** Anda sudah login sebagai **kebinganteng**	Pengguna yang telah login tidak dapat melakukan **REGISTER**
LOGOUT	>> **LOGOUT;** Anda belum login! Masuk terlebih dahulu untuk dapat mengakses Groddit. >> **LOGIN;** Masukkan *username*: **kebinganteng;** Masukkan kata sandi: **P@ssw0Rd;** Login berhasil, selamat datang di Groddit! <Tampilan Utama> >> **LOGOUT;** Anda berhasil logout. Sampai jumpa di pertemuan berikutnya!	

QUIT	>> **QUIT;** Anda telah keluar dari program Groddit.	

4. Profil

a. Perintah ini hanya dapat digunakan oleh pengguna **yang telah login**.

b. Perintah akan menampilkan profil lengkap dari pengguna dengan `<username>` yang dituju.

c. Jika pengguna dengan `<username>` yang dituju tidak ditemukan, sistem akan menampilkan pesan kesalahan.

d. Jika pengguna yang sedang login (misal: kebinganteng) mengikuti pengguna yang profilnya sedang dilihat (misal: alice), maka akan muncul status yang menyatakan apakah pengguna saat ini mengikuti pengguna yang dilihat profilnya.

e. Berikut adalah informasi yang harus ada dalam profil
   - **ID pengguna**
   - **Karma** → Karma dihitung sebagai total kumulatif dari semua upvote (+1) dikurangi semua downvote (-1) yang diterima pengguna pada semua post dan komentar yang pernah mereka buat.
   - **Tanggal akun dibuat**
   - **Jumlah Post dan Comment**
   - **Jumlah Followers dan Following**
   - **Tiga (3) Post terbaru pengguna**

Perintah	Contoh Program	Keterangan
PROFILE [USERNAME]	>> **PROFILE alice**; USER PROFILE : **alice** ----------	Melihat profil pengguna yang ada

	```ID: USER007 Karma: 1250 Account Created: 2025-10-31 21:10:00 Posts: 15 Comments: 87 Followers: 15 Following: 23 Anda mengikuti pengguna ini. Post Terbaru: 1. [r/programming] How to learn pointers (45↑) posted: 2025-11-01 21:10:00 2. [r/algorithms] BFS vs DFS explanation (32↑) posted: 2025-11-01 21:10:00 3. [r/askreddit] Best C textbooks? (18↑) posted: 2025-11-01 21:10:00 ----------``` **PROFILE [USERNAME]**	>> **PROFILE budi;** Tidak ada pengguna dengan username **budi**. Harap periksa masukan Anda!	Melihat profil pengguna yang tidak ada

5. Post

Fitur Post memungkinkan pengguna membuat, melihat, dan menghapus postingan di dalam sebuah Subgroddit (komunitas *tematik*). Post dapat menerima upvote/downvote dan menjadi tempat diskusi melalui komentar.

a. Command POST. Command ini bertujuan untuk membuat sebuah postingan baru pada suatu Subgroddit.

Perintah	Contoh Program	Keterangan

POST	`>> POST;`
	
	`Masukkan nama Subgroddit:`
	`>> r/algorithms;`
	
	`Masukkan judul post:`
	`>> Perbedaan BFS dan DFS;`
	
	`Masukkan konten post:`
	`>> Ada yang bisa jelaskan perbedaan dan kasus penggunaan BFS serta DFS?;`
	
	`Post berhasil dibuat dengan ID: P001 di Subgroddit r/algorithms!`
		Melakukan post pada *subgroddit* yang sudah ada.

POST	`>> POST;`
	
	`Masukkan nama Subgroddit:`
	`>> r/random;`
	
	`Subgroddit r/random belum ditemukan!`
	`Gunakan perintah CREATE_SUBGRODDIT terlebih dahulu untuk membuatnya.`
		Melakukan post pada *subgroddit* yang belum ada.

Untuk panjang untuk setiap input – seperti panjang nama, judul, ataupun konten – dibebaskan penentuan constraint panjang maksimalnya, namun pastikan rasional dan sesuai dengan best practice yang ada.

b. Command `VIEW_POST [ID]`. Command ini bertujuan untuk melihat detail dari suatu postingan, dimulai dari isi konten postingan, *upvote*, dan *downvote*, serta dengan komentar-komentar yang terdapat pada postingan tersebut.

Perintah	Contoh Program	Keterangan
VIEW_POST [POST_ID]	`>> VIEW_POST P001;`	Melihat *post* yang ada.

	═══════════════════════ [r/algorithms] Perbedaan BFS dan DFS oleh: kebinganteng (2025-11-02 21:10:00) ═══════════════════════ Ada yang bisa jelaskan perbedaan dan kasus penggunaan BFS serta DFS? ↑ 45 ↓ 3 Komentar: 1. [1] (alice) BFS cocok untuk shortest path! ↑ 10 ↓ 0 └── [3] (bob) Setuju, DFS lebih untuk eksplorasi semua kemungkinan. ↑ 7 ↓ 1 2. [2] (drneroifa) Gunakan queue pada BFS dan stack pada DFS. ↑ 12 ↓ 0 ═══════════════	
VIEW_POST [POST_ID]	>> **VIEW_POST P999;** Post dengan ID P999 tidak ditemukan!	Melihat *post* yang tidak ada.

c. Command DELETE_POST [ID]. Command ini bertujuan untuk menghapus sebuah postingan. Dengan catatan:

   i. Postingan **HANYA** dapat dihapus oleh pemilik postingan itu sendiri.

   ii. Jika sebuah postingan dihapus, maka komentar-komentar serta informasi yang berhubungan dengan postingan juga perlu dihapus pula (***CASCADING DELETE***).

Perintah	Contoh Program	Keterangan
DELETE_POST [POST_ID]	>> **DELETE_POST P001;** Apakah Anda yakin ingin menghapus post [P001]	Pengguna menghapus *post* miliknya.

	"Perbedaan BFS dan DFS"? (Y/N) >> **Y;** Post [P001] dan seluruh komentar terkait berhasil dihapus.	
DELETE_POST [POST_ID]	>> **DELETE_POST P002;** Anda bukan pembuat post [P002]! Hanya pembuat post yang dapat menghapus postingan ini.	Pengguna mencoba menghapus *post* yang bukan miliknya.
DELETE_POST [POST_ID]	>> **DELETE_POST P999;** Post [P999] tidak ditemukan! Anda tidak bisa menghapus fakta bahwa dia tidak dengan mu </3!!	Pengguna mencoba menghapus *post* yang tidak ada.

6. Subgroddit

Fitur Subgroddit adalah komunitas tematik tempat pengguna dapat mengunggah dan melihat post sesuai topik tertentu. Setiap Subgroddit memiliki daftar post dan dapat diurutkan berdasarkan HOT (banyak upvote) atau NEW (tanggal terbaru).

a. Command `CREATE_SUBGRODDIT <NAME>`. Command ini bertujuan untuk membuat Subgroddit baru, dengan catatan:
   i. Nama Subgroddit **HARUS UNIK** dan diawali dengan "**r/**".
   ii. Inisialisasi subgroddit akan menyimpan nama dan daftar post kosong.

Perintah	Contoh Program	Keterangan
CREATE_SUBGRODDIT [NAME]	>> **CREATE_SUBGRODDIT r/algorithms;** Subgroddit r/algorithms berhasil dibuat!	Pengguna membuat *subgroddit* baru.

	`Kamu dapat mulai membuat post di dalamnya dengan perintah POST.`	

`CREATE_SUBGRODDIT [NAME]`	`>>` **`CREATE_SUBGRODDIT r/programming;`**
	`Subgroddit r/programming berhasil dibuat!`
	
	`>>` **`CREATE_SUBGRODDIT r/programming;`**
	`Subgroddit r/programming sudah ada!`
	`Gunakan nama lain untuk Subgroddit baru.`
		Pengguna mencoba membuat *subgroddit* yang sudah dibuat sebelumnya.

b. Command `VIEW_SUBGRODDIT <NAME> <MODE> <ORDER>`. Command ini bertujuan untuk melihat seluruh postingan yang terdapat pada suatu subgroddit tertentu, dengan ketentuan.

   i. NAME merupakan nama dari subgroddit yang hendak dilihat detailnya atau *view*.

   ii. MODE merupakan *flag* untuk menentukan untuk mengurutkan postingan berdasarkan:

      1. HOT (diurutkan berdasarkan banyaknya upvote)
      2. NEW (diurutkan berdasarkan tanggal pembuatan)

   iii. ORDER merupakan *flag* untuk menentukan urutan tampilan post.

      1. INCR (dari urutan rendah ke tinggi)
      2. DECR (dari urutan tinggi ke rendah)

Perintah	Contoh Program	Keterangan

`VIEW_SUBGRODDIT <NAME> <MODE> <ORDER>`	`>>` **`VIEW_SUBGRODDIT r/algorithms HOT DECR;`**
	
	`═══════════════════════════`
	🔥 `Subgroddit: r/algorithms (sorted by HOT ↓)`
	`═══════════════════════════`
	`1. [P003] Optimasi Dijkstra vs A* (87↑ / 2↓) - oleh drneroifa`	

	2. [P001] Perbedaan BFS dan DFS (45↑ / 3↓) - oleh kebinganteng 3. [P002] Cara kerja Topological Sort (28↑ / 1↓) - oleh alice ═══════════════════ Gunakan VIEW_POST <ID> untuk melihat detail postingan.	
VIEW_SUBGRODDIT <NAME> <MODE> <ORDER>	>> **VIEW_SUBGRODDIT r/algorithms NEW INCR;** ═══════════ 🕓 Subgroddit: r/algorithms (sorted by NEW ↑) ═══════════ 1. [P001] Perbedaan BFS dan DFS (2025-11-01 21:10:00) - oleh kebinganteng 2. [P002] Cara kerja Topological Sort (2025-11-02 21:10:00) - oleh alice 3. [P003] Optimasi Dijkstra vs A* (2025-11-03 21:10:00) - oleh drneroifa ═══════════ Gunakan VIEW_POST <ID> untuk melihat detail postingan.	
VIEW_SUBGRODDIT <NAME> <MODE> <ORDER>	>> **VIEW_SUBGRODDIT r/random HOT DECR;** Subgroddit r/random belum ditemukan! Gunakan perintah CREATE_SUBGRODDIT untuk membuatnya terlebih dahulu.	
VIEW_SUBGRODDIT <NAME> <MODE> <ORDER>	>> **VIEW_SUBGRODDIT r/algorithms COOL INCR;** Mode COOL tidak dikenali! Gunakan HOT atau NEW sebagai mode yang valid.	

7. Comment and Reply

a. Fitur ini memungkinkan pengguna menulis komentar pada sebuah post, serta memberikan balasan (*reply*) terhadap komentar lain.

b. Fitur Utama :

- `COMMENT <POST_ID> <COMMENT_ID>`

  Pengguna dapat menulis komentar baru langsung ke post jika COMMENT_ID = -1, atau membalas komentar lain jika COMMENT_ID berisi ID komentar tertentu.

- `DELETE_COMMENT <POST_ID> <COMMENT_ID>`

  Pengguna **hanya dapat menghapus komentarnya sendiri** menggunakan perintah `DELETE_COMMENT`, di mana penghapusan bersifat *cascading*, artinya semua balasan di bawah komentar tersebut akan ikut terhapus.

Perintah	Contoh Program	Keterangan
COMMENT <POST_ID> <COMMENT_ID>	>> **COMMENT P001 -1;** Masukkan isi komentar: **Wah, tubesnya asik banget!;** Komentar berhasil ditambahkan ke post P001.	Pengguna menambahkan komentar pada sebuah post.
COMMENT <POST_ID> <COMMENT_ID>	>> **COMMENT P001 3;** Masukkan isi komentar: >> Setuju banget, **tubes alprog paling seru!**; Balasan berhasil ditambahkan ke komentar #3.	Pengguna menambahkan komentar pada komentar lain.
DELETE_COMMENT <POST_ID> <COMMENT_ID>	**>> DELETE_COMMENT P001 2;** Komentar #2 dan semua reply di bawahnya berhasil dihapus.	Pengguna mencoba menghapus komentar yang tidak ada.

8. Voting

Fitur Voting memungkinkan pengguna memberikan apresiasi (*upvote*) atau ketidaksetujuan (*downvote*) terhadap post dan komentar di Groddit. Setiap vote akan mempengaruhi karma pemilik konten tersebut. Sistem voting bersifat dinamis, artinya pengguna dapat mengubah atau membatalkan vote mereka kapan saja.

Mekanisme Voting:

- Setiap pengguna hanya dapat memberikan **satu vote** (upvote atau downvote) per post/komentar
- Upvote menambah **+1** ke counter upvote dan **+1** ke karma pemilik konten
- Downvote menambah **+1** ke counter downvote dan **-1** ke karma pemilik konten
- Pengguna dapat mengubah vote mereka (dari upvote ke downvote atau sebaliknya)
- Pengguna dapat membatalkan vote mereka menggunakan perintah UNDO
- Pengguna **tidak dapat** memberikan vote pada konten milik mereka sendiri

Command untuk Post

Perintah	Contoh Program	Keterangan

UPVOTE_POST	`>> UPVOTE_POST P001;`
	`Anda berhasil memberikan upvote pada post [P001].`
	
	`>> UPVOTE_POST P001;`
		Memberikan upvote pada sebuah postingan.
		
		Jika sebelumnya pengguna memberikan downvote, maka

Perintah	Contoh Program	Keterangan

	`Anda sudah memberikan upvote pada post [P001].`
	
	`>> UPVOTE_POST P001;`
	`Vote Anda pada post [P001] berhasil diubah menjadi upvote.`
	
	`>> UPVOTE_POST P001;`
	`Anda tidak dapat memberikan vote pada post Anda sendiri!`
	
	`>> UPVOTE_POST P999;`
	
	`Post dengan ID P999 tidak ditemukan!`
		downvote akan dibatalkan dan diganti dengan upvote.
		
		Karma pemilik post akan bertambah +1 (atau +2 jika sebelumnya di-downvote).

DOWNVOTE_POST	`>> DOWNVOTE_POST P002;`
	`Anda berhasil memberikan downvote pada post [P002].`
	
	`>> DOWNVOTE_POST P002;`
		Memberikan downvote pada sebuah postingan.
		
		Jika sebelumnya pengguna memberikan upvote, maka upvote akan

Perintah	Contoh Program	Keterangan
	Anda sudah memberikan downvote pada post [P002]. >> **DOWNVOTE_POST P002;** Vote Anda pada post [P002] berhasil diubah menjadi downvote. >> **DOWNVOTE_POST P002;** Anda tidak dapat memberikan vote pada post Anda sendiri! >> **DOWNVOTE_POST P999;** Post dengan ID P999 tidak ditemukan!	dibatalkan dan diganti dengan downvote. Karma pemilik post akan berkurang -1 (atau -2 jika sebelumnya di-upvote).
UNDO_VOTE_POST	>> **UNDO_VOTE_POST P001;** Vote Anda pada post [P001] berhasil dibatalkan. >> **UNDO_VOTE_POST P001;**	Membatalkan vote (upvote atau downvote) yang telah diberikan pada sebuah postingan. Karma pemilik post akan dikembalikan sesuai dengan vote yang dibatalkan:

Perintah	Contoh Program	Keterangan

	`Anda belum memberikan vote pada post [P001].`
	
	`>> UNDO_VOTE_POST P999;`
	`Post dengan ID P999 tidak ditemukan!`
		- Jika membatalkan upvote, karma -1
		- Jika membatalkan downvote, karma +1

Command untuk Comment

Perintah	Contoh Program	Keterangan

`UPVOTE_COMMENT`	`>> UPVOTE_COMMENT P001 3;`
	`Anda berhasil memberikan upvote pada komentar #3 di post [P001].`
	
	`>> UPVOTE_COMMENT P001 3;`
	`Anda sudah memberikan upvote pada komentar #3 di post [P001].`
	
	`>> UPVOTE_COMMENT P001 3;`
		Memberikan upvote pada sebuah komentar.
		
		Jika sebelumnya pengguna memberikan downvote, maka downvote akan dibatalkan dan diganti dengan upvote.
		
		Karma pemilik komentar akan bertambah +1 (atau +2 jika sebelumnya di-downvote).

Perintah	Contoh Program	Keterangan
	Vote Anda pada komentar #3 di post [P001] berhasil diubah menjadi upvote. >> **UPVOTE_COMMENT P001 3;** Anda tidak dapat memberikan vote pada komentar Anda sendiri! >> **UPVOTE_COMMENT P001 999;** Komentar #999 tidak ditemukan pada post [P001]! >> **UPVOTE_COMMENT P999 1;** Post dengan ID P999 tidak ditemukan!	
DOWNVOTE_COMMENT	>> **DOWNVOTE_COMMENT P001 5;** Anda berhasil memberikan downvote pada komentar #5 di post [P001]. >> **DOWNVOTE_COMMENT P001 5;**	Memberikan downvote pada sebuah komentar. Jika sebelumnya pengguna memberikan upvote, maka upvote akan

Perintah	Contoh Program	Keterangan
	Anda sudah memberikan downvote pada komentar #5 di post [P001]. >> **DOWNVOTE_COMMENT P001 5;** Vote Anda pada komentar #5 di post [P001] berhasil diubah menjadi downvote. >> **DOWNVOTE_COMMENT P001 5;** Anda tidak dapat memberikan vote pada komentar Anda sendiri! >> **DOWNVOTE_COMMENT P001 999;** Komentar #999 tidak ditemukan pada post [P001]! >> **DOWNVOTE_COMMENT P999 1;** Post dengan ID P999 tidak ditemukan!	dibatalkan dan diganti dengan downvote. Karma pemilik komentar akan berkurang -1 (atau -2 jika sebelumnya di-upvote).

Perintah	Contoh Program	Keterangan
UNDO_VOTE_COMMENT	>> **UNDO_VOTE_COMMENT P001 3;** Vote Anda pada komentar #3 di post [P001] berhasil dibatalkan. >> **UNDO_VOTE_COMMENT P001 3;** Anda belum memberikan vote pada komentar #3 di post [P001]. >> **UNDO_VOTE_COMMENT P001 999;** Komentar #999 tidak ditemukan pada post [P001]! >> **UNDO_VOTE_COMMENT P999 1;** Post dengan ID P999 tidak ditemukan!	Membatalkan vote yang telah diberikan pada sebuah komentar. Karma pemilik komentar akan dikembalikan sesuai dengan vote yang dibatalkan: - Jika membatalkan upvote, karma -1 - Jika membatalkan downvote, karma +1

Ketentuan Penting

1. **Autentikasi**: Pengguna harus sudah login untuk dapat melakukan voting
2. **Self-voting**: Pengguna tidak dapat memberikan vote pada konten milik mereka sendiri

9. Social

a. Fitur ini menunjukkan keterhubungan antar pengguna di Groddit.

b. Pengguna dapat *follow* (*mengikuti*) pengguna lain.
c. Pengguna dapat *unfollow* (berhenti mengikuti) pengguna lain yang sudah di-*follow* sebelumnya. Lakukan validasi untuk perintah *unfollow* pengguna yang belum diikuti sebelumnya
d. Fitur sosial ini bersifat satu arah. Pengguna yang *follow* pengguna lain tidak berarti pengguna lain *follow* balik pengguna tersebut.
e. Gunakan **ADT Graf** untuk menyimpan informasi Social**.** representasi dari ADT ini dibebaskan (cth. Adjacency Matrix, Adjacency List).
f. Berikut adalah contoh masukan dan keluaran perintah yang berkaitan dengan fitur ini. (Silakan berkreativitas untuk format input/output selama informasi yang ditampilkan minimal seperti contoh)

Perintah	Contoh Program	Keterangan

FOLLOW	`>> FOLLOW alice;`
	`Anda berhasil mengikuti alice.`
	
	`>> FOLLOW alice;`
	`Anda sudah mengikuti alice.`	

UNFOLLOW	`>> UNFOLLOW alice;` `Anda berhasil berhenti mengikuti alice.`	

FOLLOWERS	`>> FOLLOWERS alice;`
	`Pengikut alice:`
	`1. kebinganteng`
	`2. drneroifa`	

FOLLOWING	`>> FOLLOWING alice;`
	`alice mengikuti:`
	`1. kebinganteng`
	`2. nimon_dave`	

10. Save & Load

a. **Save** adalah perintah untuk menyimpan kondisi (*state*) dari Groddit saat ini ke dalam suatu folder berisi seluruh berkas konfigurasi. Perintah ini bisa dilakukan pada seluruh kondisi, baik saat pengguna masuk ke dalam akun ataupun saat pengguna belum masuk ke dalam akun.

b. **Load** adalah perintah untuk memuat kondisi (*state*) dari folder berisi seluruh berkas konfigurasi ke dalam aplikasi Groddit. Perintah ini hanya bisa dilakukan saat pengguna belum masuk ke dalam akun.

c. Seluruh berkas konfigurasi akan disimpan sesuai dengan nama yang telah ada di Konfigurasi Program. Untuk melakukan pemuatan, pengguna memasukkan nama folder tempat dimana berkas konfigurasi disimpan. Seluruh berkas konfigurasi diasumsikan selalu ada secara lengkap dengan nama yang sesuai dengan yang telah ada di spesifikasi.

d. NOTES!

> **Pemuatan seluruh berkas konfigurasi wajib menggunakan ADT Mesin Kata ketika sedang melakukan *parsing*. Anda hanya diperkenankan menggunakan pustaka stdio untuk membuka *file*.**

e. Silakan *handle* kasus dimana input text dalam “” dan input text tanpa “”. Contoh: load harus dapat membaca kedua tipe input seperti berikut: **...,”ini adalah text”,...** dan **...,ini adalah text,...**

f. Berikut daftar perintah dan contoh masukan serta keluaran ketika pemanggilan perintah. Tampilan dibebaskan berikut ini hanya contoh saja.

Perintah	Contoh Program	Keterangan

SAVE	`>> SAVE;`
	
	`Masukkan nama folder penyimpanan: Folder1;`
	
	`Belum terdapat Folder1. Akan dilakukan pembuatan Folder1 terlebih dahulu.`
	
	`Mohon tunggu...`
	`1...`
	`2...`
	`3...`
	
	`Folder1 sudah berhasil dibuat.`
	
	`Anda akan melakukan penyimpanan di Folder1.`
	
	`Mohon tunggu...`
	`1...`
	`2...`
	`3...`
	
	`Penyimpanan telah berhasil dilakukan!`
	
	`>> SAVE;`
	
	`Masukkan nama folder penyimpanan`
	`Folder1;`
	
	`Anda akan melakukan penyimpanan di Folder1.`
		Pengguna saat ini: -
		
		Penyimpanan dilakukan di folder masukan.

	Mohon tunggu... 1... 2... 3... Penyimpanan telah berhasil dilakukan! >>	
LOAD	>> **LOAD;** Masukkan nama folder yang hendak dimuat. **Folder2;** Anda akan melakukan pemuatan dari Folder2. Mohon tunggu... 1... 2... 3... Pemuatan selesai! >>	Pengguna saat ini: -
	>> **LOAD;** Masukkan nama folder yang hendak dimuat. **Folder3;** Tidak ada folder yang dimaksud! >>	Pengguna saat ini: -

	`>> LOAD;`
	
	`Anda harus keluar terlebih dahulu untuk melakukan perintah ini.`
	
	`>>`
		Pengguna saat ini: Tuan Hak

Konfigurasi Program

Berikut adalah 5 konfigurasi yang wajib untuk dibuat. Konfigurasi bisa jadi berubah atau bertambah apabila mengerjakan spesifikasi bonus. Berikut lampiran contoh file konfigurasi Sample Data.

Konfigurasi Pengguna (user.csv)

Kolom	**Deskripsi**
user_id	Tersusun atas prefix “USER” diikuti dengan 3 digit angka yang merepresentasikan urutan registrasi user. e.g. “USER001” adalah user pertama yang diregistrasi
username	Berupa string dengan panjang maksimal 255 karakter. Note: Dipastikan unik
password	Berupa string dengan panjang minimal 8 karakter dan panjang maksimal 20 karakter. Note: Apabila mengerjakan bonus hash password maka restriksi panjang maksimal pada file konfigurasi dihilangkan, namun

	panjang password sebelum di hash tetap maksimal 20 karakter. Silakan sesuaikan kolom ini dengan password hasil hash.
karma	Berupa integer yang melambangkan tingkat upvote dan downvote pengguna
created_at	Tanggal akun dibuat dalam format “YYYY-MM-DD HH:MM:SS”

Konfigurasi Post (post.csv)

Kolom	**Deskripsi**
post_id	Tersusun atas prefix “P” diikuti dengan 3 digit angka yang merepresentasikan urutan post. e.g. “P001” adalah post pertama
subgroddit_id	Berupa string yang me-reference ke subgroddit_id pada Konfigurasi Subgroddit. “S001” berarti post tersebut *belong to* subgroddit dengan subgroddit_id “S001”
author_id	Berupa string yang me-reference ke user_id pada Konfigurasi User. “USER001” berarti post tersebut ditulis oleh user dengan user_id “USER001”
title	Berupa string dengan panjang maksimal karakter 255
content	Berupa string
created_at	Tanggal post dibuat dalam format “YYYY-MM-DD HH:MM:SS”
upvotes	Jumlah upvote pada post tersebut dalam integer

downvotes	Jumlah downvote pada post tersebut dalam integer

Konfigurasi Komentar (comment.csv)

Kolom	**Deskripsi**
comment_id	Tersusun atas integer yang merepresentasikan urutan komentar. e.g. 1 adalah komentar pertama
post_id	Berupa string yang me-reference ke post_id pada Konfigurasi Post. “P001” berarti komentar tersebut *belong to* post dengan user_id “P001”
author_id	Berupa string yang me-reference ke user_id pada Konfigurasi User. “USER001” berarti komentar tersebut ditulis oleh user dengan user_id “USER001”
parent_comment_id	Berupa string yang me-reference ke comment_id lain. 1 → berarti komentar tersebut adalah komentar terhadap komentar lain dengan comment_id 1. -1 → berarti komentar tersebut bukanlah komentar terhadap komentar lain Note: Parent_comment_id harus menunjuk pada comment dengan post_id yang sama.
content	Berupa string
upvotes	Jumlah upvote pada komentar tersebut dalam integer
downvotes	Jumlah downvote pada komentar tersebut dalam integer

Konfigurasi Subgroddit (subgroddit.csv)

Kolom	**Deskripsi**
subgroddit_id	Tersusun atas prefix “S” diikuti dengan 3 digit angka yang merepresentasikan urutan subgroddit. e.g. “S001” adalah subgroddit pertama
name	Berupa string dengan panjang maksimal 255 karakter. Note: Harus diawali dengan “r/”

Konfigurasi Relasi Sosial (social.csv)

Kolom	**Deskripsi**
user_id	Berupa string yang me-reference ke user_id pada Konfigurasi User. “USER001” berarti user dengan user_id “USER001”
following_id	Berupa string yang me-reference ke user_id pada Konfigurasi User. “USER002” berarti user pada kolom user_id sedang mem-follow user “USER002” Note: user_id tidak akan sama dengan following_id

Konfigurasi Voting (voting.csv)

Kolom	Deskripsi
user_id	User yang memberikan vote
target_type	Jenis target vote: "POST" atau "COMMENT"
target_id	ID dari post atau comment yang di-vote
vote_type	

Bonus

Terdapat beberapa bonus implementasi pada aplikasi. Implementasi ini **tidak wajib dikerjakan**, tetapi *nice to have* dan dapat menambah penilaian aplikasi Anda. Silakan prioritaskan pengerjaan *requirements* wajib terlebih dahulu sebelum memutuskan untuk mengerjakan bonus.

1. Feed

a. Fitur Feed berfungsi untuk menampilkan postingan yang dibuat oleh pengguna lain yang telah di-*follow* oleh seorang pengguna.

b. Jika pengguna belum mengikuti siapa pun, maka halaman feed akan kosong dan tidak menampilkan postingan apa pun.

c. Feed menampilkan post terbaru dari akun yang diikuti, diurutkan menurut waktu publikasi.

d. Implementasi fitur ini **wajib menggunakan** [ADT Heap] yang merupakan *tree-based data structure*.

e. Perintah utama pada fitur ini adalah SHOW_FEED [LATEST/NEWEST] [LIMIT]

- `LATEST/NEWEST` menentukan urutan tampilan feed.
  - `LATEST`: menampilkan postingan berdasarkan waktu paling baru (urutan menurun).
  - `NEWEST`: menampilkan postingan berdasarkan waktu paling lama (urutan menaik).
- `LIMIT` menyatakan banyak *post* paling banyak yang ditampilkan. Jika jumlah *post* yang tersedia kurang dari `LIMIT`, tampilkan semua *post* yang ada.

```
>> SHOW_FEED LATEST 5;

═══════════════════════════════════════
📰 Feed: alice (sorted by LATEST — newest first)
═══════════════════════════════════════
1. [P010] Menyusun grafik DAG (2025-11-08 19:22) - oleh drneroifa
```

```
2. [P009] Tips optimasi memori pada C++ (2025-11-08 18:50) - oleh bob
3. [P007] Implementasi Max Heap pada Feed Groddit (2025-11-08 17:05) - oleh charlie
4. [P005] Tutorial Redis cache singkat (2025-11-07 20:30) - oleh diana
5. [P003] Perbedaan BFS dan DFS (2025-11-07 18:10) - oleh kebinganteng
═══════════════════════════════════════════

Gunakan VIEW_POST <ID> untuk melihat detail postingan.
```

```
>> SHOW_FEED LATEST;

═══════════════════════════════════════════
📰  Feed: bob (sorted by LATEST — newest first)
═══════════════════════════════════════════
Feed kosong. Kamu belum mengikuti siapa pun.
═══════════════════════════════════════════
```

2. Trending Analysis

Fitur ini berfungsi untuk melakukan **analisis topik yang sedang tren** berdasarkan frekuensi kemunculan kata kunci (*keyword frequency*) pada judul dan isi dari postingan maupun komentar di dalam Subgroddit tertentu. Dengan adanya fitur ini, pengguna dapat melihat topik-topik apa saja yang paling sering dibicarakan dalam rentang waktu tertentu (misalnya 24 jam terakhir). Perintah yang digunakan adalah **TRENDING <subgroddit> <waktu> <hour/minute/second>**. Berikut adalah contoh dari fitur *trending analysis*.

```
>> TRENDING r/programming 24 hour
╔═══════════════════════════════════════════════════════════╗
║      TRENDING IN r/programming (Last 24 hours)            ║
╚═══════════════════════════════════════════════════════════╝
```

```
1. "pointer"        - 45 mentions
2. "memory"         - 38 mentions
3. "segfault"       - 32 mentions
4. "struct"         - 28 mentions
5. "malloc"         - 24 mentions

Hottest topic: "pointer" in "Understanding pointer arithmetic"
```

```
>> TRENDING r/programming 10 minute
╔═══════════════════════════════════════════════════════════╗
║      TRENDING IN r/programming (Last 10 minutes)          ║
╚═══════════════════════════════════════════════════════════╝
1. "pointer"        - 45 mentions
2. "memory"         - 38 mentions
3. "segfault"       - 32 mentions
4. "struct"         - 28 mentions
5. "malloc"         - 24 mentions

Hottest topic: "pointer" in "Understanding pointer arithmetic"
```

```
>> TRENDING r/programming 24 hour

Subgroddit r/programming belum ditemukan!
Gunakan perintah CREATE_SUBGRODDIT untuk membuatnya terlebih dahulu.
```

3. Advanced Search

Fitur **Advanced Search** memungkinkan pengguna melakukan pencarian terhadap **username**, **judul postingan**, maupun **nama Subgroddit** menggunakan pendekatan berbasis **ADT Trie** **(prefix tree)**. Dengan fitur ini, pengguna dapat menemukan entitas yang diinginkan bahkan hanya dengan mengetik sebagian awal kata. Misalnya, saat

pengguna ingin mencari akun dengan username “kebinganteng”, cukup mengetik keb dan sistem akan menampilkan semua pengguna yang diawali dengan kata tersebut. Berikut adalah contoh dari fitur advanced search.

```
>> SEARCH_USER keb;
Menampilkan hasil pencarian user dengan prefix "keb"...

1. kebinganteng
2. kebablicious
3. Kebobot
```

```
>> SEARCH_POST BFS;
Menampilkan hasil pencarian post dengan prefix "BFS"...

1. [r/algorithms] BFS vs DFS Explanation
2. [r/algorithms] BFS Shortest Path Tutorial
3. [r/programming] BFS in C
```

```
>> SEARCH_SUBGRODDIT r/a;
Menampilkan hasil pencarian subgroddit untuk prefix "r/a"...

1. r/algorithms
2. r/asknimon
3. r/ai_research
```

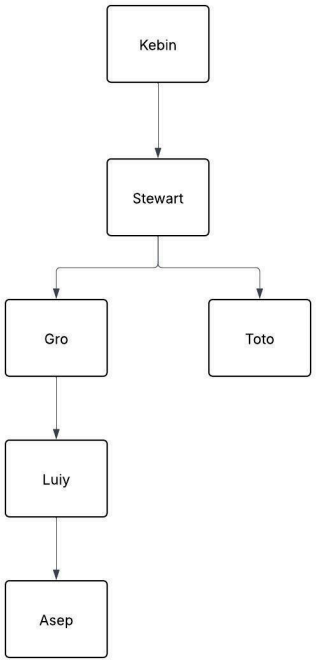
4. Following Recommendation

Fitur Following Recommendation menggunakan pendekatan Breadth-First Search (BFS) dengan kedalaman maksimum 2–3 dari current_user. Dari hasil tersebut, sistem mengumpulkan daftar pengguna yang belum di-follow oleh current_user, namun memiliki hubungan *friends of friends*. Setiap calon rekomendasi kemudian diberi peringkat

berdasarkan jumlah *mutual connections* yang dimiliki dengan current_user. Akhirnya, sistem akan menampilkan 10 pengguna teratas sebagai hasil rekomendasi teman yang paling relevan.

```
>> FRIEND_RECOMMENDATION;
Menampilkan rekomendasi teman untuk pengguna...

1. Gro (5 mutual connections)
2. Luiy (3 mutual connections)
3. Toto (2 mutual connections)
4. KDM (2 mutual connections)
```



Misalkan pengguna yang saat ini sedang login adalah Kebin. Kebin memiliki teman bernama Stewart. Berdasarkan hubungan pertemanan tersebut, pengguna yang direkomendasikan sebagai teman baru Kebin

```
adalah Gro, Luiy, dan Toto
```

5. Content Moderation

Fitur Content Moderation berfungsi untuk menjaga lingkungan komunitas agar tetap positif dan bebas dari kata-kata yang tidak pantas. Sistem ini bekerja dengan cara melakukan penyaringan otomatis terhadap kata-kata terlarang. Jika sebuah post atau komentar yang ingin diunggah mengandung kata yang terlarang, sistem akan menolak unggahan tersebut. Dengan demikian, hanya konten yang sesuai dengan pedoman komunitas yang akan dipublikasikan. List kata terlarang disimpan di file konfigurasi.

```
Contoh isi file konfigurasi

{
  "blacklisted_words": ["bodoh", "sampah", "jelek", "kasar"]
}
```

```
>> COMMENT P001 -1;
Masukkan isi komentar: Wah, postingan ini bodoh banget!;
Komentar ditolak karena mengandung kata terlarang: "bodoh".
```

```
>> POST;

Masukkan nama Subgroddit:
>> r/algorithms;

Masukkan judul post:
>> Algoritma paling jelek;

Masukkan konten post:
```

```
>> Algoritma ini sampah banget, gak berguna sama sekali!;

Post gagal dibuat karena mengandung kata terlarang: "jelek, sampah".
```

6. Data Security

Seiring berkembangnya Groddit sebagai pusat komunikasi bagi para villain, muncul satu masalah yang bahkan Dr. Neroifa sendiri tidak perkirakan—kebocoran data rahasia. Beberapa villain diketahui mencoba membuka berkas konfigurasi Groddit untuk mengetahui password rahasia Gro, atau bahkan menyalin seluruh isi Subgroddit. Gro yang panik segera memerintahkan Dr. Neroifa untuk memperkuat sistem keamanan Groddit. Setelah serangkaian percobaan, lahirlah fitur baru: Data Security System yaitu mekanisme untuk keamanan ganda yang melindungi kata sandi pengguna dan berkas penyimpanan Groddit.

Fitur Data Security bertujuan untuk memastikan keamanan data pada aplikasi Groddit melalui dua mekanisme utama:

a. **Password Hashing** – menjaga kerahasiaan kata sandi pengguna dengan menyimpannya dalam bentuk nilai hash, bukan teks asli.
b. **File Encryption & Decryption** – melindungi seluruh data hasil save agar tidak dapat dibaca secara langsung oleh pihak luar.

Kombinasi kedua mekanisme ini membuat Groddit menjadi lebih aman, meskipun jatuh ke tangan Nimon yang terlalu penasaran atau hacker dari VillainCon.

Untuk mendapatkan nilai penuh dari spesifikasi bonus ini, **Anda harus mengimplementasikan kedua mekanisme** tersebut. Algoritma hashing dan enkripsi/dekripsi yang digunakan dibebaskan. Namun hal yang penting yang perlu diperhatikan adalah ~~**kedua mekanisme tersebut harus bisa diaktifkan dan dinonaktifkan.**~~ sebagai berikut.

a. Password hashing bisa aktif maupun tidak aktif dalam suatu konfigurasi program. Namun ketika fitur password hashing telah aktif di suatu konfigurasi program, maka fitur tersebut tidak akan bisa dinonaktifkan.
b. Fitur file encryption & decryption bisa diaktifkan maupun dinonaktifkan.

7. Kreativitas

Setelah sistem keamanan Groddit diperkuat dengan enkripsi ganda oleh Dr. Neroifa, Gro menyadari satu hal: meskipun aman, tampilan antarmuka Groddit sangat membosankan. Para Nimon mulai mengeluh karena bosan melihat tampilan terminal hitam-putih yang penuh teks, dan beberapa bahkan menempelkan stiker pisang di layar agar "lebih ceria". Melihat kekacauan tersebut, Gro akhirnya mengumumkan proyek baru bernama **"Creative Initiative"**, sebuah upaya untuk membuat Groddit lebih hidup, interaktif, dan menyenangkan tanpa mengorbankan fungsionalitas.

Fitur **Creativity** memberikan kebebasan bagi pengembang Groddit untuk menambahkan fitur tambahan. Fitur ini bersifat **terbuka**. Beberapa contoh implementasi yang bisa kalian buat adalah sebagai berikut.

a. **TUI (Text User Interface)**: membuat tampilan interaktif berbasis teks menggunakan warna, frame, atau komponen navigasi terminal.
b. **Fitur Interaktif Tambahan**: misalnya efek animasi pada loading, *banner ASCII art*, sistem notifikasi visual, atau dashboard statistik.
c. **Inovasi Bebas**: fitur unik yang masih relevan dengan konsep sosial Groddit, seperti quiz komunitas, penghargaan "Nimon of the Month", atau sistem badge untuk pengguna aktif.

Batasan

Hanya diperbolehkan menggunakan pustaka eksternal sebagai berikut.

- Math (untuk operasi matematika seperti pangkat, akar, dsb. jika diperlukan)
- Stdio (untuk melakukan print, open, dan kebutuhan *unit testing*)
- Stdlib (untuk malloc dan free)
- Time (untuk mendapatkan waktu lokal)
- Sys/stat (untuk melakukan validasi terkait filesystem OS)

Gunakan **mesin karakter** dan **mesin kata** untuk melakukan manipulasi/operasi string. Jika tidak ada, silahkan implementasikan sendiri.

Saran

Pada bagian ini, hanya dituliskan **saran-saran** yang sekiranya bisa membantu mempermudah pengerjaan. **Sifat dari bagian ini adalah <u>TIDAK WAJIB</u>.** Silakan diikuti atas dasar kemauan kalian sendiri.

1. Saran *Timeline* Pengerjaan

   Mengacu pada rencana kuliah IF2110 Algoritma dan Pemrograman 2, secara umum, materi yang telah diajarkan ketika waktu rilis tugas besar adalah semua ADT yang akan digunakan pada tugas besar ini, kecuali **ADT Mesin Abstrak, Tree, dan Graph**. Oleh karena itu, berikut saran *timeline* untuk pengerjaan tugas besar.

   Catatan: Anda tidak perlu mengikuti timeline berikut. Ini hanya merupakan saran.

Kegiatan	**Minggu Perkuliahan**			
	W11	W12	W13	W14
Merancang ADT				
ADT Sederhana				
ADT List Berkait				
Mesin Karakter dan Mesin Kata				
ADT Tree				

ADT Graph				
Laporan				